

Introduction to Computer Architecture

The 80/20 Revision Guide

Part IB — Cambridge CST

How this guide is organised

The Computer Architecture paper (Paper 5) sets three questions per year. Across **four years of past papers (2022–2025, 12 questions)** — the course is newer than most — the topic distribution is:

Topic	Appearances	Share
Cache design & memory hierarchy	4	33%
Multicore & parallel processing (incl. coherence, sync)	4	33%
Pipelining & hazards (incl. microarchitecture)	4	33%
SystemVerilog & FPGA / digital design	4	33%
RISC-V ISA & load-store architecture	3	25%
DRAM operations	2	17%
Virtual memory / memory protection	2	17%
Branch and hazard handling	1	8%
Exceptions & coprocessors	1	8%
Amdahl's law & performance	1	8%
Power efficiency & dark silicon	1	8%
Side-channel attacks / hardware security	1	8%

80/20 verdict. Caches, pipelining, multicore (with cache coherence), and SystemVerilog account for roughly 80% of marks. The pattern of the paper is: question (a) is digital-design / SystemVerilog, question (b) is microarchitecture-flavoured (pipeline or cache), question (c) is the system-level paper-stretcher (multicore, virtual memory, security).

This document goes in that order: **must-know first, then important, then context.**

1 The Big Four (80% of marks)

1.1 Caches and the memory hierarchy

Exam-critical

This is the single most consistent question topic. Expect: “calculate the miss rate / CPI / cache size for these parameters,” “compare direct-mapped, set-associative, fully-associative,” “explain write-through vs write-back,” “classify these misses as compulsory / capacity / conflict.”

Why a hierarchy?

Physics: at 5 GHz, light travels only 60 mm in one clock cycle, and electrical signals on chip <20 mm. A large memory cannot also be low-latency. Solution: layer fast-small over slow-big and let **locality** make up the difference.

- **Temporal locality:** a word accessed now is likely to be accessed again soon (loops, repeated variables).

- **Spatial locality:** words near a recent access are likely to be accessed soon (sequential code, array walks).

Typical numbers to know:

Level	Latency	Size
L1 SRAM (on-chip)	1–9 cycles	tens of KiB
L2 SRAM	~20 cycles	256 KiB–1 MiB
L3 SRAM (LLC)	~40 cycles	MiBs–tens of MiB
DRAM (off-chip)	~100 cycles	GiBs
Disk / SSD	ms / μ s	TiBs

Cache anatomy and the address split

A direct-mapped cache splits the address into three fields:

tag	index	offset
upper bits	which line	byte/word in line

Worked sizing example. 32-bit address, 32 KiB cache, 64-byte lines, direct-mapped.

- **Offset:** $\log_2 64 = 6$ bits.
- **Number of lines:** $32 \text{ KiB} / 64 = 512$. **Index:** $\log_2 512 = 9$ bits.
- **Tag:** $32 - 9 - 6 = 17$ bits.

You will be asked this. Drill it.

Three placement policies

- **Direct-mapped:** each address maps to exactly one cache line. Cheap, fast, prone to conflict misses.
- **Fully associative:** an address can live anywhere; needs N parallel tag comparators. Best hit rate, biggest hardware cost, scarcely used beyond TLBs and victim caches.
- **N -way set associative:** the common compromise. The cache is split into sets; each set has N candidate slots; the index selects the set, then N tag comparators check the set in parallel. Typical: 4–16 ways.

Write policies

- **Write-through:** write to cache and to next level. Always consistent; slow without a **write buffer** (which lets the core continue while writes drain).
- **Write-back:** write only to the cache; mark the line **dirty**. On eviction of a dirty line, write to next level. Faster on average, more complex.
- **Write-allocate** vs **write-around** on a write miss: do we fetch the line into cache, or write past it?

The common pairing: write-back + write-allocate. The other common pairing: write-through + no-allocate.

The three C's of misses

- **Compulsory (cold):** first access to a block. Reduced by larger lines / prefetching.
- **Capacity:** working set bigger than cache. Reduced by a bigger cache.
- **Conflict:** multiple addresses competing for the same set. Reduced by higher associativity. *Wouldn't happen in a fully-associative cache.*

The standard CPI/AMAT calculation

Exam-critical

This appears almost every year. Memorise the formulas.

Average Memory Access Time: $AMAT = t_{hit} + r_{miss} \cdot t_{penalty}$.

Effective CPI (one cache):

$$CPI_{eff} = CPI_{base} + r_{miss/instr} \cdot t_{penalty} \text{ (cycles)}$$

Worked example (Patterson & Hennessy). Base CPI = 1, clock = 4 GHz, miss rate/instruction = 2%, main memory access = 100 ns.

- Miss penalty = $100 \text{ ns} / 0.25 \text{ ns/cycle} = 400 \text{ cycles}$.
- $CPI = 1 + 0.02 \times 400 = 9$.

Add an L2: access time 5 ns, global miss rate to DRAM = 0.5%.

- L1-miss-L2-hit penalty = 20 cycles. L1-miss-L2-miss extra penalty = 400.
- $CPI = 1 + 0.02 \times 20 + 0.005 \times 400 = 3.4$. Speedup = $9 / 3.4 \approx 2.6$.

Cache design trade-off table (memorise)

Change	Effect on miss rate	Cost
Larger cache	Capacity misses ↓	Access time ↑
Higher associativity	Conflict misses ↓	Access time ↑, hardware ↑
Larger line size	Compulsory misses ↓, then ↑ (pollution)	Miss penalty ↑
Add a victim cache	Some conflict misses caught	Tiny extra hardware

1.2 Pipelining and hazards

Exam-critical

Two question patterns: (i) “draw a pipeline diagram for this code, mark stalls and forwarding paths”; (ii) “compare branch prediction strategies”; (iii) “calculate speedup of pipelined vs single-cycle.” Always relative to the canonical **5-stage RISC-V pipeline**.

The 5 stages

1. **IF** — instruction fetch from I-memory
2. **ID** — instruction decode, register file read

3. **EX** — ALU operation or address calculation
4. **MEM** — data memory access (load/store only)
5. **WB** — write result back to register file

Speedup intuition. If all stages take the same time T , a non-pipelined cycle takes $5T$ but the pipelined throughput is one instruction per T . Maximum speedup = number of stages, in practice less due to stalls and unbalanced stages.

The three hazards

- **Structural** — two instructions want the same resource at once. Cured by duplicating resources (separate I-cache and D-cache so IF and MEM don't fight for memory).
- **Data** — an instruction needs the result of one still in flight.
- **Control** — the next PC depends on a branch we haven't resolved yet.

Solving data hazards: forwarding (bypassing)

The result of the EX stage is available at the end of EX, even though it hasn't been written back. **Forwarding** routes the EX output (or MEM output for loads) directly to the EX input of a following instruction.

Producer stage finishes at	Available to next-EX from
EX (ALU op)	end of EX $\Rightarrow$ next-cycle EX input
MEM (load)	end of MEM $\Rightarrow$ one bubble before next EX

Load-use hazard. Even with full forwarding, you cannot forward from MEM to the immediately following EX ("forward backward in time"). You must stall one cycle, or reorder the code:

```
# Original (causes one stall):
lw  x1, 0(sp)
lw  x2, 8(sp)
add x3, x1, x2  # uses x2 immediately - STALL
sw  x3, 24(sp)

# Reordered (no stall):
lw  x1, 0(sp)
lw  x2, 8(sp)
lw  x4, 16(sp)  # filler load
add x3, x1, x2
sw  x3, 24(sp)
```

Solving control hazards: prediction

Stall on branch is correct but expensive — one bubble per branch on a 5-stage pipeline, several on a deeper one.

- **Static prediction:** "backward branches taken, forward branches not taken" — exploits the fact that loop branches usually go back.
- **Dynamic prediction:** a branch predictor records the recent outcome (one-bit or two-bit saturating counter per branch entry). ~95% accuracy on real workloads.
- **On mispredict:** squash the in-flight wrong-path instructions and restart fetch from the correct PC.

1.3 Multicore: coherence, consistency, synchronisation

Exam-critical

The system-level question often lives here. You should be able to (a) walk the MSI/MESI protocol state machine for a specific sequence of reads and writes across cores, (b) explain why caches need a coherence protocol, (c) write a correct spin-lock using LR/SC, and (d) recognise when memory-ordering subtlety matters.

Cache coherence: the problem

Multiple cores, private L1 caches, shared memory. If core 1 holds $X = 5$ in its L1 and core 2 writes $X = 7$, core 1's view becomes stale. Coherence makes the caches **behave as if there were a single shared memory**.

Snoopy bus. The cheap, classic mechanism: all caches sit on a broadcast bus and observe (*snoop*) every other cache's transactions. Each cache line carries a **state** per coherence protocol.

MSI protocol

Three states per line:

- **M (Modified):** only this cache has it; value differs from memory. Implicit “dirty.”
- **S (Shared):** this cache has a clean copy; other caches may too.
- **I (Invalid):** no valid copy here.

Transitions (the most testable):

Action	From	To state & effect
Local read	I	S; BusRd; if another cache has it in M, that cache writes back and goes to S
Local read	S, M	same; no bus traffic
Local write	I	M; BusRdX (read-with-intent-to-modify); other caches invalidate
Local write	S	M; Invalidate broadcast; others go to I
Local write	M	same; no bus traffic
Snoop BusRd of my M-line	M	S; write back
Snoop BusRdX/Inv of my S or M	→	I

MESI adds an *Exclusive* state for “I have the only clean copy”; lets you go E→M without bus traffic. **MOESI adds Owned** for shared-but-dirty (one cache holds the only up-to-date copy that others also share, replied to instead of memory).

Memory consistency

Coherence settles *when* writes to one location become visible. Consistency settles *the order in which writes to different locations are observed*. Compilers and hardware reorder loads/stores — the classic example:

```
// Initially A = B = 0
Core 1:          Core 2:
A = 1;           B = 1;
if (B == 0) X = 3;  if (A == 0) Y = 5;
```

Sequential-consistent answer: at most one of $X = 3$ or $Y = 5$ can hold. **In practice on modern relaxed memory models, both can happen** because the read of B (resp. A) can be reordered before the write. Cure: **memory fence** (fence rw,rw) between the write and the read.

Atomic primitives: LR/SC on RISC-V

Load-Reserved / Store-Conditional replaces test-and-set. The store-conditional succeeds only if no other write to the same line happened in the interval; otherwise the SC fails and you retry.

```
# Efficient spin lock: read until it looks free, only then attempt SC
lock: lr.w    x4, (x1)      # load-reserved
      bne     x4, zero, lock # is lock taken? spin (no writes!)
      li      x3, 1
      sc.w    x3, x3, (x1)   # store-conditional
      bne     x3, zero, lock # SC failed - retry
      # ... critical section ...
unlock:
      sw      zero, 0(x1)
```

Why the inner-loop read matters. A naive “test-and-set” that writes every iteration thrashes the cache coherence protocol — every core’s spin pulls the lock cache line into M state, evicting it from everyone else. The version above only reads in the spin (line stays in S across all spinners), and only attempts the write when the lock looks free.

1.4 Digital design: SystemVerilog & FPGAs

Exam-critical

The opening question on most papers. You will be asked to write or read SystemVerilog for a small synchronous circuit (a counter, a multiplexer, a small state machine, a piece of a datapath), and to reason about FPGA mapping (LUTs, flip-flops, block RAM, DSP).

Synchronous design fundamentals

- **Clocked elements** are D flip-flops; combinational logic between them computes the next state.
- **Setup time:** input D must be stable for t_{su} *before* the clock edge.
- **Hold time:** D must remain stable for t_h *after* the edge.
- **Critical path** = the longest combinational delay between any two registers. Sets the maximum clock frequency: $f_{\max} = 1/(t_{cq} + t_{\text{logic}} + t_{su})$.

SystemVerilog skeleton

```
module counter2bit(
    input logic    clk,
    input logic    rst_n,
    input logic    go,
    output logic [1:0] n);

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n)    n <= 2'b00;
        else if (go)    n <= n + 2'b01;

endmodule
```

The three always blocks to know:

- `always_ff @(posedge clk)` — synthesises to registers (flip-flops). Use `<=` (non-blocking).
- `always_comb` — combinational logic. Use `=` (blocking). Every output must be assigned on every path or you create unwanted latches.
- `always_latch` — explicit level-sensitive latch. Avoid except for tools that need it.

Blocking vs non-blocking. In `always_ff`, `a <= b; b <= a;` swaps both atomically (non-blocking: RHS evaluated first, LHS updated at clock edge). With `a = b; b = a;` you'd just set both to the old `b`.

A small finite state machine

```
typedef enum logic [1:0] { IDLE, BUSY, DONE } state_t;
state_t state, next_state;

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else       state <= next_state;

always_comb begin
    next_state = state;           // default to hold
    case (state)
        IDLE: if (start) next_state = BUSY;
        BUSY: if (done)  next_state = DONE;
        DONE:           next_state = IDLE;
    endcase
end
```

The two-process FSM pattern: one `always_ff` for state register, one `always_comb` for next-state and outputs. Easier to debug, no implicit latches.

FPGA resources

An FPGA is roughly a sea of:

- **LUTs** (Look-Up Tables) — typically 4–6 input, each a tiny programmable truth table. Implement combinational logic.
- **Flip-flops** attached to LUTs — state.
- **Block RAM (BRAM)** — dedicated SRAM blocks, ~kilobits each. Used for caches, FIFOs, register files.
- **DSP blocks** — hard-wired multiply-accumulate units, for ML and signal processing.
- **Routing fabric** — the programmable interconnect, where most of the area and delay live.

Implication for caches: a direct-mapped cache maps neatly onto BRAM (one read port, one write port, sequential). A fully-associative cache requires N tag comparators in parallel — on an FPGA those have to be built from registers + LUTs, blowing up area. *This is the FPGA-specific reason set-associative is the common compromise.*

2 The Second Tier

2.1 RISC-V ISA basics

Worth knowing

You don't need to memorise every instruction. You do need: register conventions, the four instruction formats, the basic operations, and **the RISC philosophy** of orthogonal, fixed-length, load-store ISA.

The RISC philosophy in three bullets

- **Load-store architecture.** Arithmetic operates only on registers; memory is touched only by `lw/sw`.
- **Fixed-length, regular encoding.** Easy fetch and decode in one cycle. RV32: every base instruction is 32 bits.
- **Few, orthogonal instructions.** All ALU ops take the same operand layout; registers are general-purpose.

Register file (RV32I)

32 integer registers `x0–x31`. `x0` is hard-wired to zero. The ABI assigns roles:

Reg	ABI	Role
<code>x0</code>	zero	constant 0
<code>x1</code>	ra	return address (caller-saved)
<code>x2</code>	sp	stack pointer (callee-saved)
<code>x5--x7, x28--x31</code>	t0--t6	temporaries (caller-saved)
<code>x8, x9, x18--x27</code>	s0--s11	saved (callee-saved)
<code>x10--x17</code>	a0--a7	args / return values (caller-saved)

Common instructions

```
add  x3, x2, x1    # x3 = x2 + x1
addi x3, x3, 1     # x3 = x3 + 1
lw   x4, 0(x3)     # x4 = MEM[x3 + 0]
sw   x4, 8(x3)     # MEM[x3 + 8] = x4
beq  x4, x5, label # if x4 == x5, branch to label
jal  ra, foo       # ra = PC+4; PC = foo (call)
jalr zero, ra, 0   # PC = ra (return)
```

CISC vs RISC

RISC (RISC-V, ARM)	CISC (x86)
Fixed-length instructions (32-bit)	Variable-length (1–17 bytes)
Load-store: only <code>lw/sw</code> touch memory	Many instructions can have memory operands
Few, orthogonal ops (~50 in base ISA)	Hundreds of instructions, many redundant
Easy to pipeline and decode in parallel	Hard — modern x86 decodes to micro-ops internally
Dominant in mobile, embedded, accelerators	Dominant in PC & server for backward compatibility

2.2 Virtual memory

Worth knowing

Appears in 2022-5-8, 2024-5-8. Expect: “explain TLB, page table walker, page fault handler”; “compute the address translation from this multi-level page table.”

The mapping

The processor produces a **virtual address**; hardware translates to a **physical address**. Translation is at the granularity of a **page** (typically 4 KiB) or a **superpage** (e.g. 4 MiB).

Address split for a 4 KiB page on a 32-bit virtual address: low 12 bits are the page offset (unchanged), high 20 bits are the virtual page number (VPN), translated to a physical page number (PPN).

Page table

An array of **Page Table Entries (PTEs)**, indexed by VPN. Each PTE holds the PPN plus permission/status bits (R, W, X, Valid, Dirty, Accessed, User, Global).

A flat 32-bit / 4 KiB scheme would need $2^{20} \times 4 \text{ B} = 4 \text{ MiB}$ per process — mostly empty. Solution: **multi-level page tables**. On RISC-V Sv32: the 20-bit VPN splits into two 10-bit pieces; the upper 10 indexes into a top-level table whose entries either point to a leaf page or to a second-level table. Each sub-table is one page.

TLB — the cache of PTEs

A translation per memory access would double DRAM bandwidth and add hundreds of cycles. The **TLB** caches recent PTEs (16–512 entries, fully or highly associative). Typical: 1 cycle hit, 10–100 cycles miss.

TLB miss. Either (a) hardware page-table walker reads memory, fills the TLB, retries; or (b) raise a software exception and let the OS do it (older designs). Modern CPUs use hardware walkers.

Page fault. If the PTE says “not present,” the OS handles it: locate the page on disk, possibly evict a dirty page (write-back!), read in, update the PTE, restart the faulting instruction. Can cost *millions* of cycles.

Memory protection

The OS gives each process its own page table. On context switch, change the page-table base register (**satp** on RISC-V). The **ASID** (address-space ID) is also held in TLB entries so a context switch doesn’t have to flush the whole TLB. The OS lives in **Supervisor mode**; user processes in **User mode**; privileged instructions and registers (**satp**, **mtvec**, **sepc**) are accessible only from privileged modes.

2.3 Exceptions and interrupts

Worth knowing

Specifically tested in 2025-5-7. Be able to distinguish exception (synchronous, instruction-caused), interrupt (asynchronous, external) and software exception (**ecall**).

- **Exceptions** are synchronous to the instruction stream: divide-by-zero, illegal instruction, page fault, misaligned access.

- **Interrupts** are asynchronous: timer, DMA-complete, I/O device. The current instruction completes; the handler runs.
- **Software exceptions** (`ecall`, `ebreak`) are deliberate transitions into a more privileged mode — system calls.

RISC-V mechanism. On any trap: save current PC to `sepc` (or `mepc`), put cause in `scause`, jump to handler at `stvec`. Handler completes; `sret` restores PC and privilege.

2.4 Amdahl’s and Gustafson’s laws

Worth knowing

Tested in 2023-5-7. One-formula questions; learn the formulas and a sentence on each.

Amdahl (fixed problem, scale cores). For sequential fraction B and n cores:

$$\text{Speedup}(n) = \frac{1}{B + (1 - B)/n}$$

Implication. As $n \rightarrow \infty$, $\text{Speedup} \rightarrow 1/B$. If 10% is sequential, you cap at 10× speedup, no matter how many cores. *The sequential bottleneck dominates.*

Gustafson (fixed time, scale problem). $\text{Speedup}(n) = n - (n - 1)B$. The optimistic counterpart: with more cores you tackle bigger problems, and the parallel fraction grows.

When examiners ask you to “critique”: Amdahl assumes the problem is fixed; Gustafson assumes time is fixed. Real workloads sit between these two extremes.

3 The Third Tier (context & flavour)

3.1 The eight great ideas in computer architecture

A useful mnemonic for “what is computer architecture about” — comes straight from Patterson & Hennessy:

1. Design for Moore’s law (assume next year’s chip has $\sim 2\times$ transistors).
2. Use abstraction to simplify design (HLL $\rightarrow$ assembly $\rightarrow$ machine code).
3. Make the common case fast.
4. Performance via parallelism (ILP, DLP, TLP).
5. Performance via pipelining (one production line per stage).
6. Performance via prediction (branch, cache, value).
7. Hierarchy of memories (caches).
8. Dependability via redundancy (ECC, RAID, lockstep cores).

3.2 Performance equation

$$\text{CPU time} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

What changes what:

Affects	Instr count	CPI	Clock period
Program	•	•	
Compiler	•	•	
ISA	•	•	•
Microarchitecture		•	•
Implementation tech			•

3.3 DRAM operation (one-paragraph version)

A DRAM chip is a 2D array of bit-cells, each a single capacitor. An access:

1. **Row activate** (RAS): the row address is decoded; the entire row is pulled into a row buffer, destroying the row's contents in the array.
2. **Column access** (CAS): one or more columns are read from the row buffer.
3. **Precharge**: the row buffer is written back to the array, restoring it.

This is why DRAM is **burst-oriented** (read many words from one activated row, cheap) and **row-locality-sensitive** (different row $\Rightarrow$ full activate/precharge cycle, expensive). DDR4 typically does 8 bursts of 64 bits = 64 B per transfer — not coincidentally, the standard cache line size.

DRAM cells must be refreshed (typically every ~ 64 ms) because charges leak. The refresh controller schedules these without disrupting normal access.

3.4 Flynn's taxonomy

	Single instr	Multi instr
Single data	SISD (classic CPU)	MISD (rare; redundancy)
Multi data	SIMD (vectors, GPUs)	MIMD (multicore)

Note SIMT (Single Instruction, Multiple Thread, used by GPUs) is essentially SIMD with the appearance of independent threads; the hardware groups threads into **warps** and executes them in lockstep.

3.5 Dark silicon and the energy wall

Transistors keep shrinking, but **Dennard scaling** (voltage scaling with size) ended around 2005. Today you can fit more transistors on a chip than you can power simultaneously — the unpowered ones are **dark silicon**. Consequence: clock speeds plateaued; instead, vendors added more cores and **accelerators** (TPU, GPU, NIC offload, crypto engines, codecs). Energy efficiency, not transistor count, is now the limiter.

3.6 Side-channel attacks (Spectre)

Worth knowing

Specifically tested in 2025-5-8. Be able to explain the mechanism in 3 sentences.

Mechanism. Speculative execution lets a CPU run instructions past a branch before knowing whether the branch is taken. If the branch turns out mispredicted, those instructions are squashed — *architecturally*. But they still left side effects in the **cache**. By timing subsequent loads, an attacker can tell which addresses were touched, leaking secret-dependent information.

Spectre example sketch:

```

uint8_t A[10], B[256 * 64];
void victim(size_t i) {
    if (i < 10) {
        uint8_t v = A[i];           // mispredicted as taken
        uint8_t junk = B[64 * v];   // out-of-bounds read; speculative
    }
}
// Attacker times reads of B and infers v from which line is now warm.

```

Mitigations. Speculation barriers (**lfence**), retpolines, hiding speculative cache state (InvisiSpec), and tracking taint (STT). All have a performance cost.

4 The one-page exam checklist

Exam-critical

If you have one hour to revise, do these in order:

1. Be able to compute tag/index/offset bit widths from cache size, associativity, line size, address width.
2. Be able to compute AMAT and effective CPI given miss rates and penalties — one cache level and two.
3. Be able to classify a sequence of accesses into compulsory / capacity / conflict misses.
4. Be able to explain write-through vs write-back, write-allocate vs no-allocate, and the rationale for the common pairings.
5. Be able to draw the 5-stage RISC-V pipeline, identify a data hazard in given assembly, and show where forwarding paths go.
6. Be able to detect a load-use hazard and either insert a stall bubble or reorder code to remove it.
7. Be able to walk MSI (or MESI) through a worked sequence of reads/writes across two cores.
8. Be able to write a correct LR/SC spin-lock and explain why naive test-and-set thrashes coherence.
9. Be able to write a small SystemVerilog FSM (two-process pattern) without creating implicit latches.
10. Know which FPGA resource maps to what: LUT → combinational, FF → state, BRAM → cache/FIFO, DSP → MAC.
11. Be able to walk a multi-level page table translation and explain TLB hits, TLB misses, page faults.
12. Be able to state Amdahl's law, derive the asymptotic speedup, and contrast with Gustafson.
13. Be able to explain Spectre in three sentences (speculative execution + cache side channel + timing).

Good luck.